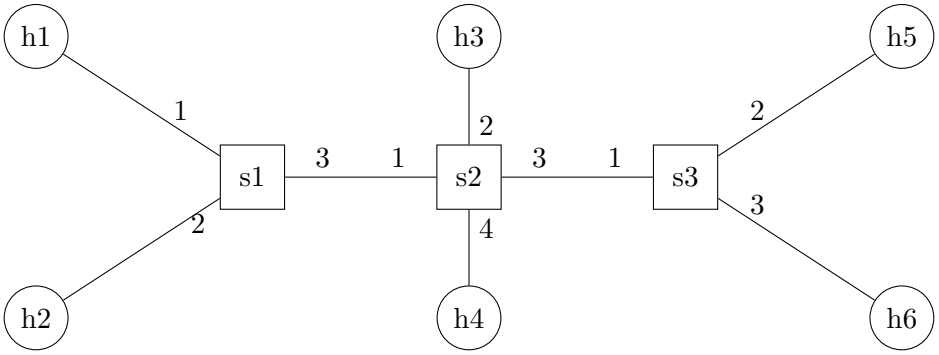
Software Defined Networking
21 giugno 2022

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm. La funzione di gestione dell'evento PacketIn è data nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura. Se non altrimenti specificato, il TTL iniziale è 200.



Negli switch sono preinstallate le regole in Tabella 1.

L'host h1 porta 3322 invia un datagramma UDP verso h4 porta 123.

1. Scrivere se $s2$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

   Soluzione: Non manda pacchetti al controllore.

2. Scrivere tutti gli host che ricevono il pacchetto in uscita da s2. Per ogni host scrivere il numero di porta UDP destinazione e il TTL.

   Soluzione: h3, h4. porta 123. TTL 200

L'host h1 porta 333 apre una connessione TCP verso h4 porta 123.

3. Scrivere se $s1$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

   Soluzione: Sì, invia una PacketIn e riceve una PacketOut

4. Scrivere se $s2$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

Soluzione: Sì, invia una PacketIn e riceve una FlowMod

5. Scrivere tutti gli host che ricevono il pacchetto in uscita da s2. Per ogni host scrivere il numero di porta TCP destinazione e il TTL.

Soluzione: s2 non invia pacchetti

L'host h1 ritrasmette il pacchetto della precedente apertura di connessione.

6. Scrivere se $s2$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

Soluzione: Non manda pacchetti al controllore.

7. Scrivere tutti gli host che ricevono il pacchetto in uscita da s2. Per ogni host scrivere il numero di porta TCP destinazione e il TTL.

Soluzione: Nessun host, il pacchetto viene mandato in s3.

Il destinatario dell'apertura di connessione invia un pacchetto di risposta.

8. Scrivere gli indirizzi IP sorgente e destinazione e il TTL del pacchetto all'ingresso dello switch s2.

Soluzione: IP sorgente = IP5, IP destinazione = IP1, TTL = 200

9. Dire se e quali regole devono essere aggiunte alla tabela del nodo s2 per consentire al pacchetto di risposta di raggiungere il client corretto.

Soluzione: Va aggiunta una regola che cambi IP sorgente da IP5 a IP4.

Match: eth_type=IPv4, ipv4_src=IP5, ipv4_dst=IP1, ip_proto=TCP, tcp_src=123, tcp_dst=123

Action: ip_src=IP4, output(1)

Tabella 1: Regole iniziali

Switch s1

Priority	Match	Action
5	*	Output(FLOOD)
10	eth_type = IPv4 ip_proto=TCP	Output(CONTROLLER)
15	eth_dst = MAC2	DecTTL, Output(2)
15	eth_dst = MAC1	DecTTL, Output(1)

Switch s2

Priority	Match	Action
0	*	Output(FLOOD)
10	eth_type = IPv4 ip_proto = TCP	DecTTL Output(CONTROLLER)
10	eth_type = IPv4 ip_proto = UDP	Output(FLOOD)

Switch s3

Priority	Match	Action
0	*	Output(FLOOD)
15	eth_dst = MAC5	DecTTL, Output(2)
15	eth_dst = MAC6	DecTTL, Output(3)

Programma 1 – Packet In

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg; datapath = msg.datapath; ofproto = datapath.ofproto;
    parser = datapath.ofproto_parser; in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data); pkt_eth = pkt.get_protocol(eth.eth)
    pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
    pkt_udp = pkt.get_protocol(udp.udp); pkt_tcp = pkt.get_protocol(tcp.tcp)
    data = msg.data

    if (datapath.id == 2 and ipv4_dst == IP4):
        match = parser.OFPMatch(eth_type=ETH_TYPE_IP,
            ipv4_dst = IP4)
        actions = [
            parser.OFPSetField(ipv4_dst = IP5),
            parser.OFPDecTTL(),
            parser.OFPActionOutput(3) ]
        inst = [ parser.OFPInstructionActions(
            ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=20,
            match=match, instructions=inst)
        datapath.send_msg(mod)
        return

    actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
    out = parser.OFPPacketOut(datapath=datapath,data=msg.data,
                in_port=in_port, actions=actions)
    datapath.send_msg(out)
```

Esercizio 2

Si consideri una rete con 4 router di frontiera identificati dalle lettere A, B, C e D. Il traffico in entrata e in uscita da ogni router è riportato nella seguente tabella. I pacchetti sono tutti di lunghezza $L = 500$ ottetti.

Router	Traffico IN (Mbit/s)	Traffico OUT (Mbit/s)
A	10	20
B	25	35
C	25	15
D	30	20

10. Usando il *metodo gravitazionale*, stimare il traffico tra l'ingresso A e l'uscita C.

Soluzione:

$$\text{Traffico} = \frac{A_{IN} \times C_{OUT}}{A_{OUT} + B_{OUT} + C_{OUT} + D_{OUT}} = \frac{10 \times 15}{90} = 1{,}66\,\text{Mbit/s}$$

11. La misura del traffico dall'ingresso B avviene campionando i pacchetti con probabilità $p = 0.01$. Qual è la probabilità che non si osservi neanche un pacchetto di una connessione di $N = 50$ pacchetti?

Soluzione:

$$\Pr\{\text{nessun pacchetto}\} = (1-p)^N \simeq e^{-pN} = 0.6$$

12. Per una connessione TCP sono stati osservati $x = 10$ pacchetti. Qual è la migliore stima del volume di dati della connessione?

Soluzione:

$$\hat{V} = \frac{xL}{p} = 500\,\text{kB}$$

13. Da ogni pacchetto vengono estratti i campi che definiscono la connessione TCP; su di essi è calcolato un hash. La più lunga sequenza di zeri iniziali osservata nell'output della funzione hash è $z = 10$. Qual è la migliore stima del numero totale di connessioni?

Soluzione:

$$\#\hat{C} = \frac{2^{z+1}}{\phi} = 2648$$

Esercizio 3

Un router di frontiera conserva una black list di indirizzi IP i cui pacchetti devono essere scartati. Il router usa un filtro di Bloom *a conteggio* costituito da un array di $m = 2^{16}$ celle e $k = 4$ funzioni hash. Gli indirizzi bloccati sono $n = 15\,000$.

Nel filtro a conteggio, l'array anziché essere binario è di numeri interi che sono incrementati ad ogni inserimento.

14. Calcolare la probabilità che un pacchetto legittimo sia scartato.

Soluzione:

$$\left(1 - \exp\left(-\frac{kn}{m}\right)\right)^k = 0.13$$

15. Calcolare la somma dei valori delle celle dell'array.

Soluzione: Ogni elemento inserito aggiunge k al conteggio.

$$nk = 60\,000$$

16. Si supponga che $w = 1000$ elementi siano rimossi dal filtro. Qual è la probabilità che il filtro sia corretto dopo le rimozioni?

Soluzione: Ad ogni rimozione si corre il rischio di corrompere il filtro quando si rimuove un elemento che non era nel filtro (falso positivo)

La probabilità cercata è la probabilità di non incappare mai in un falso positivo. Usando la probabilità di falso positivo trovata precedentemente

$$\Pr\{\text{filtro corretto}\} = (1 - 0.13)^{1000} \simeq 0$$

Nota: questa è una stima per difetto, perché in realtà la probabilità di falso positivo scende man mano che elementi sono rimossi dal filtro, ma anche usando la probabilità di falso positivo finale il risultato rimane prossimo a zero.

Si consideri la famiglia di funzioni hash

$$h_i(x) = 3ix \bmod 1024 \quad \text{per } 1 \leq i \leq 4$$

17. Inizialmente l'array è vuoto. Inserire nel filtro gli elementi 1 e 3. Mostrare lo stato finale del sistema.

Soluzione: Quando inseriamo $x = 1$, aggiungiamo uno agli elementi: 3, 6, 9, 12. Quando inseriamo $x = 3$, aggiungiamo uno agli elementi: 9, 18, 27, 36.

Il filtro finale avrà tutti zero tranne:

$$\{3 \to 1,\ 6 \to 1,\ 9 \to 2,\ 12 \to 1,\ 18 \to 1,\ 27 \to 1,\ 36 \to 1\}$$

18. Spiegare perché funzioni $h_2(x)$ e $h_4(x)$ sono o non sono buone funzioni hash.

Soluzione:

Le due funzioni non sono suriettive. Infatti entrambe danno come risultato un numero inferiore a 1024, mentre il codominio è l'intervallo dei numeri interi tra 0 (incluso) e 2^{16} (escluso). Inoltre $h_2(x)$ dà come risultato solo numeri pari e $h_4(x)$ dà come risultato solo multipli di 4.